

A cybersteps project

Date: 15. May 2026

Participants: Gregor

Executive Summary

This project implements an automated security posture analyzer for a Microsoft Entra ID (Azure AD) tenant. A Python script connects to a lab tenant via the Microsoft Graph API, collects identity and access data, and sends it to an Azure OpenAI service for AI-driven security analysis. The result is a streamed security report with concrete findings and remediation recommendations, printed directly to the terminal.

How the Script Works

The script runs in four sequential stages:

1. Authentication

The script uses the MSAL library to authenticate to the lab tenant using the OAuth 2.0 client credentials flow (App Registration with a client secret). This returns a bearer token scoped to Microsoft Graph, which is used for all subsequent API calls.

2. Data Collection

Five data sets are pulled from Microsoft Graph using paginated GET requests (following @odata.nextLink automatically):

- Users — display name, UPN, account status, license assignments, on-premises sync status
- Directory Roles — all active roles and their current members
- Conditional Access Policies — state, conditions, grant controls, and session controls
- MFA Registration — per-user authentication methods with a two-stage approach:
 - Stage 1: bulk beta endpoint (fast, requires P1/P2 license)
 - Stage 2: per-user /authentication/methods fallback (free tier compatible)
- Groups — group type, security settings, and member lists (capped at 20 per group)

3. AI Analysis

The collected data is serialized to JSON and sent to an Azure OpenAI GPT-4 deployment via the OpenAI Python SDK. A system prompt instructs the model to act as a senior Entra ID security architect and produce a structured report covering users, privileged

roles, Conditional Access, MFA, and groups. The response is streamed back and printed to the terminal in real time.

4. Output

The report is printed directly to the terminal as it streams. It includes an executive summary, severity-rated findings per category (CRITICAL / HIGH / MEDIUM / LOW), specific references to user principal names and policy names from the tenant data, and a prioritized list of remediation actions.

Tenant Configuration

Lab Tenant — App Registration

The following steps were completed in the lab tenant to allow the script to read identity data via Microsoft Graph:

Step 1: Create the App Registration

- Navigate to Entra ID → App registrations → New registration
- Name: SecurityAnalyzer (or any name)
- Supported account types: Single tenant
- Redirect URI: leave blank
- Click Register

Step 2: Note the IDs

- Copy the Application (client) ID → LAB_CLIENT_ID in .env
- Copy the Directory (tenant) ID → LAB_TENANT_ID in .env

Step 3: Create a Client Secret

- Go to Certificates & secrets → New client secret
- Set a description and expiry, click Add
- Copy the secret value immediately → LAB_CLIENT_SECRET in .env

Step 4: Add API Permissions

Add the following Application permissions under Microsoft Graph, then click Grant admin consent:

Permission	Purpose
User.Read.All	Read all users and their properties

Permission	Purpose
RoleManagement.Read.All	Read directory roles and their members
Policy.Read.All	Read Conditional Access policies
UserAuthenticationMethod.Read.All	Read per-user MFA/auth method data
Group.Read.All	Read groups and their memberships

Important: All permissions must be Application permissions (not Delegated), and admin consent must be granted for each one. Permissions show as pending (yellow) until consent is explicitly

Main Tenant — Azure OpenAI

The following steps were completed in the main tenant to provision the Azure OpenAI service used for analysis:

Step 1: Create the Azure OpenAI Resource

- In the Azure portal, create a new Azure OpenAI resource in your main tenant
- Choose a region with available GPT-4 quota (East US or Sweden Central recommended)

Step 2: Deploy a Model

- Go to ai.azure.com (Azure AI Foundry) and select your resource
- Navigate to Deployments → Deploy model → Deploy base model
- Select gpt-4o and give it a deployment name (e.g. gpt-4o)
- Wait 1–2 minutes for provisioning to complete

Step 3: Collect Configuration Values

- Endpoint URL (e.g. [https://\(resource\).openai.azure.com/](https://(resource).openai.azure.com/)) → AOAI_ENDPOINT
- API key from Keys and Endpoint → AOAI_API_KEY
- Exact deployment name from the Deployments list → AOAI_DEPLOYMENT

Environment Variables (.env)

All configuration is loaded from a .env file in the project directory:

Variable	Description
LAB_TENANT_ID	Directory (tenant) ID of the lab tenant
LAB_CLIENT_ID	Application (client) ID of the app registration
LAB_CLIENT_SECRET	Client secret value from the app registration

Variable	Description
AOAI_ENDPOINT	Full URL of the Azure OpenAI resource
AOAI_API_KEY	API key for the Azure OpenAI resource
AOAI_DEPLOYMENT	Exact deployment name (e.g. gpt-4o)
AOAI_API_VERSION	API version string (default: 2024-12-01-preview)

Running the Script

```
pip install msal requests openai python-dotenv
python entra_security_analyzer.py
```

The resulting audit report

Executive Summary

The Microsoft Entra ID (Azure AD) tenant has several critical security gaps that need immediate attention. The most pressing issues include a lack of Conditional Access policies, insufficient Multi-Factor Authentication (MFA) coverage, over-privileged Global Administrators, and potential risks associated with group configurations. These vulnerabilities expose the tenant to risks such as unauthorized access, privilege escalation, and data breaches.

Findings by Category

Users

Findings:

- **Stale Accounts**:** No disabled or stale accounts were identified. However, service accounts like "Automate Bot" and "Conf Room" accounts (e.g., ``Adams@WWLx879667.0nMicrosoft.com``) are active without MFA.
 - **Severity**:** MEDIUM
- **External Users**:** No external users were identified in the tenant.
 - **Severity**:** LOW
- **Password Hygiene**:** All users rely on passwords, and many accounts lack MFA registration.
 - **Severity**:** HIGH

Recommendations:

- Disable unused service accounts or enforce MFA for them.
- Regularly review and disable stale accounts.
- Implement password policies to enforce complexity and expiration.

Privileged Roles

Findings:

1. ****Over-Privileged Accounts****: Six users are assigned the ****Global Administrator**** role, which is excessive for most organizations.
 - Users: `admin@WWLx879667.onmicrosoft.com`,
`AllanD@WWLx879667.OnMicrosoft.com`, `NestorW@WWLx879667.OnMicrosoft.com`,
`IsaiahL@WWLx879667.OnMicrosoft.com`, `MeganB@WWLx879667.OnMicrosoft.com`,
`LidiaH@WWLx879667.OnMicrosoft.com`.
 - ****Severity****: CRITICAL
2. ****Orphaned Assignments****: No orphaned role assignments were identified.
 - ****Severity****: LOW

Recommendations:

- Reduce the number of Global Administrators to a maximum of 2–3 users.
 - Implement Privileged Identity Management (PIM) to enforce just-in-time (JIT) access for privileged roles.
 - Regularly audit role assignments.
-

Conditional Access

Findings:

1. ****No Conditional Access Policies****: The tenant lacks Conditional Access policies, leaving it vulnerable to unauthorized access and brute-force attacks.
 - ****Severity****: CRITICAL
2. ****No MFA Enforcement****: MFA is not enforced for any users, including privileged accounts.
 - ****Severity****: CRITICAL

Recommendations:

- Create Conditional Access policies to enforce MFA for all users, especially for privileged roles.
 - Implement location-based restrictions to block access from high-risk regions.
 - Require compliant devices for access to sensitive resources.
-

MFA Registration

Findings:

1. ****Low MFA Coverage****: Only two users (`AdeleV@WWLx879667.OnMicrosoft.com` and `admin@WWLx879667.onmicrosoft.com`) have MFA enabled.
 - ****Severity****: CRITICAL

2. ****Weak Default MFA Methods****: The default MFA method for all users is "Password," which is not secure.

– ****Severity****: HIGH

Recommendations:

- Enforce MFA registration for all users.
- Set a secure default MFA method, such as Microsoft Authenticator or FIDO2 security keys.
- Use Conditional Access policies to block access for users without MFA registration.

Groups

Findings:

1. ****Security Groups with Privileged Members****: Some security groups (e.g., `sg-IT`, `sg-Sales and Marketing`) contain privileged users, increasing the risk of privilege escalation.

– ****Severity****: HIGH

2. ****Empty Groups****: Groups like `sg-HR` have no members, which may indicate poor group management.

– ****Severity****: LOW

3. ****Mail-Enabled Groups Without Security****: Groups like `Sales Team` and `Finance Team` are mail-enabled but not security-enabled, which could lead to misconfigurations.

– ****Severity****: MEDIUM

Recommendations:

- Review and restrict membership of security groups with privileged users.
- Delete or repurpose empty groups.
- Ensure mail-enabled groups are security-enabled if they are used for access control.

Top Priority Actions

1. ****Enforce MFA for All Users****:

- Create a Conditional Access policy to enforce MFA for all users, starting with privileged accounts.
- Ensure MFA registration is completed for all users.

2. ****Reduce Global Administrators****:

- Remove unnecessary Global Administrator assignments. Retain only essential users, such as `admin@WWLx879667.onmicrosoft.com`.

3. ****Implement Conditional Access Policies****:

- Create policies to enforce MFA, block legacy authentication, and restrict access from high-risk locations.

4. ****Enable Privileged Identity Management (PIM)****:

- Use PIM to enforce just-in-time access for privileged roles and require MFA for role activation.

5. ****Audit and Secure Groups****:

- Review group memberships, especially for security groups with privileged users.
- Remove unnecessary members and enforce least privilege principles.

6. ****Secure Service Accounts****:

- Enforce MFA for service accounts like ``AutomateB@WWLx879667.0nMicrosoft.com``.
- Use app registrations or managed identities for automation tasks instead of service accounts.

By addressing these critical issues, the tenant's security posture will be significantly improved, reducing the risk of unauthorized access and data breaches.

Done.

Source Code of the python script

```
"""
```

```
Entra ID Security Posture Analyzer
```

```
=====
```

```
Pulls data from a lab tenant via Microsoft Graph API and sends it  
to Azure OpenAI for security analysis and recommendations.
```

```
Requirements:
```

```
pip install msal requests openai python-dotenv
```

Setup:

Copy `.env.example` to `.env` and fill in your values.

.....

```
import json
```

```
import os
```

```
import sys
```

```
import textwrap
```

```
from typing import Any
```

```
import requests
```

```
from dotenv import load_dotenv
```

```
from msal import ConfidentialClientApplication
```

```
from openai import AzureOpenAI
```

```
load_dotenv()
```

```
# -----
```

```
# Configuration (loaded from .env)
```

```
# -----
```

```
# Lab tenant – Microsoft Graph
```

```
LAB_TENANT_ID = os.getenv("LAB_TENANT_ID")
```

```

LAB_CLIENT_ID = os.getenv("LAB_CLIENT_ID")

LAB_CLIENT_SECRET = os.getenv("LAB_CLIENT_SECRET")


# Your main tenant – Azure OpenAI

AOAI_ENDPOINT = os.getenv("AOAI_ENDPOINT") # e.g.
https://<name>.openai.azure.com/

AOAI_API_KEY = os.getenv("AOAI_API_KEY")

AOAI_DEPLOYMENT = os.getenv("AOAI_DEPLOYMENT") # e.g. gpt-4o

AOAI_API_VERSION = os.getenv("AOAI_API_VERSION", "2024-12-01-preview")


GRAPH_BASE = "https://graph.microsoft.com/v1.0"

GRAPH_BETA = "https://graph.microsoft.com/beta"

GRAPH_SCOPES = ["https://graph.microsoft.com/.default"]


# -----

# Helpers

# -----


def _check_env() -> None:

    required = {

        "LAB_TENANT_ID": LAB_TENANT_ID,

        "LAB_CLIENT_ID": LAB_CLIENT_ID,

        "LAB_CLIENT_SECRET": LAB_CLIENT_SECRET,

```

```

"AOAI_ENDPOINT": AOAI_ENDPOINT,

"AOAI_API_KEY": AOAI_API_KEY,

"AOAI_DEPLOYMENT": AOAI_DEPLOYMENT,

}

missing = [k for k, v in required.items() if not v]

if missing:

    print(f"[ERROR] Missing environment variables: {' ', '.join(missing)}")

    print(" Copy .env.example to .env and fill in your values.")

    sys.exit(1)


def _section(title: str) -> None:

    width = 60

    print(f"\n{'-' * width}")

    print(f" {title}")

    print(f"{'-' * width}")


def _get_graph_token() -> str:

    """Acquire a Graph API access token using client credentials."""

    app = ConfidentialClientApplication(

        client_id=LAB_CLIENT_ID,

        client_credential=LAB_CLIENT_SECRET,

        authority=f"https://login.microsoftonline.com/{LAB_TENANT_ID}",

```

```

)

result = app.acquire_token_for_client(scopes=GRAPH_SCOPES)

if "access_token" not in result:

    error = result.get("error_description", result.get("error", "Unknown"))

    print(f"[ERROR] Failed to acquire Graph token: {error}")

    sys.exit(1)

    print("[✓] Graph API token acquired")

    return result["access_token"]


def _graph_get(token: str, url: str, params: dict | None = None) ->
list[dict]:

    """

    Paginated GET against Microsoft Graph.

    Follows @odata.nextLink automatically.

    """

    headers = {"Authorization": f"Bearer {token}", "Accept": "application/json"}

    results: list[dict] = []

    next_url: str | None = url

    while next_url:

        resp = requests.get(next_url, headers=headers, params=params, timeout=30)

        if resp.status_code == 403:

            print(f" [!] 403 Forbidden – missing Graph permission for: {url}")

```

```

return []

resp.raise_for_status()

data = resp.json()

# Some endpoints return a single object, not a list

if "value" in data:

    results.extend(data["value"])

else:

    return [data]

next_url = data.get("@odata.nextLink")

params = None # params are encoded into nextLink already


return results


# -----

# Data collection

# -----


def collect_users(token: str) -> list[dict]:

    _section("Collecting Users")

    users = _graph_get(

        token,

        f"{GRAPH_BASE}/users",

        params={

```



```

"userPrincipalName": m.get("userPrincipalName"),

"userType": m.get("userType"),

}

for m in members

],

})

print(f" Found {len(enriched)} active directory roles")

return enriched


def collect_conditional_access(token: str) -> list[dict]:

    _section("Collecting Conditional Access Policies")

    policies = _graph_get(token, f"
{GRAPH_BASE}/identity/conditionalAccess/policies")

    simplified = []

    for p in policies:

        simplified.append({

            "displayName": p.get("displayName"),

            "state": p.get("state"), # enabled / disabled /
enabledForReportingButNotEnforced

            "conditions_users": p.get("conditions", {}).get("users", {}),

            "conditions_applications": p.get("conditions", {}).get("applications", {}),

            "grantControls": p.get("grantControls"),

            "sessionControls": p.get("sessionControls"),

```

```
})
```

```
print(f" Found {len(simplified)} Conditional Access policies")
```

```
return simplified
```

```
def collect_mfa_registration(token: str) -> list[dict]:
```

```
.....
```

Two-stage approach:

1. Try the beta bulk endpoint (requires P1/P2 license).

2. On 403, fall back to per-user /authentication/methods (free tier compatible).

Required app permission: UserAuthenticationMethod.Read.All

```
.....
```

```
_section("Collecting MFA Registration Status")
```

```
# — Stage 1: bulk endpoint (fast, license-gated) —————
```

```
headers = {"Authorization": f"Bearer {token}", "Accept": "application/json"}
```

```
resp = requests.get(
```

```
f"{GRAPH_BETA}/reports/authenticationMethods/userRegistrationDetails",
```

```
headers=headers,
```

```
timeout=30,
```

```
)
```

```

if resp.status_code == 200:

    records = resp.json().get("value", [])

    simplified = [

        {

            "userPrincipalName": r.get("userPrincipalName"),

            "isMfaRegistered": r.get("isMfaRegistered"),

            "isMfaCapable": r.get("isMfaCapable"),

            "isSsprRegistered": r.get("isSsprRegistered"),

            "isPasswordlessCapable": r.get("isPasswordlessCapable"),

            "methodsRegistered": r.get("methodsRegistered", []),

            "defaultMfaMethod": r.get("defaultMfaMethod"),

        }

        for r in records

    ]

    print(f" Found MFA records for {len(simplified)} users (bulk endpoint)")

    return simplified


# — Stage 2: per-user fallback —————

print(" Bulk endpoint unavailable (likely no P1/P2 license) – falling back
to per-user method lookup...")


# Fetch all users first (reuse existing Graph helper)

users = _graph_get(

    token,

```

```

f"{GRAPH_BASE}/users",

params={"$select": "id,userPrincipalName,displayName"},

)

# Method type → human-readable label mapping

MFA_METHODS = {

"#microsoft.graph.microsoftAuthenticatorAuthenticationMethod":
"MicrosoftAuthenticator",

"#microsoft.graph.phoneAuthenticationMethod": "Phone/SMS",

"#microsoft.graph.fido2AuthenticationMethod": "FIDO2/Passkey",

"#microsoft.graph.windowsHelloForBusinessAuthenticationMethod": "WindowsHello",

"#microsoft.graph.emailAuthenticationMethod": "Email",

"#microsoft.graph.passwordAuthenticationMethod": "Password",

"#microsoft.graph.softwareOathAuthenticationMethod": "TOTP/SoftwareOATH",

"#microsoft.graph.temporaryAccessPassAuthenticationMethod":
"TemporaryAccessPass",

}

# Methods that count as MFA-capable (i.e. stronger than just password)

MFA_CAPABLE_TYPES = {

"#microsoft.graph.microsoftAuthenticatorAuthenticationMethod",

"#microsoft.graph.fido2AuthenticationMethod",

"#microsoft.graph.windowsHelloForBusinessAuthenticationMethod",

```

```
"#microsoft.graph.phoneAuthenticationMethod",  
  
"#microsoft.graph.softwareOathAuthenticationMethod",  
  
"#microsoft.graph.temporaryAccessPassAuthenticationMethod",  
  
}
```

```
results = []
```

```
total = len(users)
```

```
for i, user in enumerate(users, start=1):
```

```
    uid = user.get("id")
```

```
    upn = user.get("userPrincipalName", "unknown")
```

```
    print(f" [{i}/{total}] {upn}", end="\r")
```

```
    methods_raw = _graph_get(token, f"  
    {GRAPH_BASE}/users/{uid}/authentication/methods")
```

```
    method_types = [m.get("@odata.type", "") for m in methods_raw]
```

```
    method_labels = [  
    MFA_METHODS.get(t, t.split(".")[1]) for t in method_types
```

```
    ]
```

```
# A user is MFA-capable if they have at least one non-password strong method
```

```
mfa_capable = any(t in MFA_CAPABLE_TYPES for t in method_types)
```

```
# Password-only = not MFA registered
```

```
non_password = [t for t in method_types if t !=
```

```

"#microsoft.graph.passwordAuthenticationMethod"]

mfa_registered = len(non_password) > 0


results.append({

"userPrincipalName": upn,

"isMfaRegistered": mfa_registered,

"isMfaCapable": mfa_capable,

"methodsRegistered": method_labels,

"defaultMfaMethod": method_labels[0] if method_labels else None,

})


print() # clear the \r line

print(f" Found MFA records for {len(results)} users (per-user fallback)")

return results


def collect_groups(token: str) -> list[dict]:

_section("Collecting Groups & Memberships")

groups = _graph_get(

token,

f"{GRAPH_BASE}/groups",

params={"$select":

"id,displayName,groupTypes,securityEnabled,mailEnabled,description"},

)

```

```

enriched = []

for g in groups:

members = _graph_get(token, f"{GRAPH_BASE}/groups/{g['id']}/members")

enriched.append({

"displayName": g.get("displayName"),

"groupTypes": g.get("groupTypes", []),

"securityEnabled": g.get("securityEnabled"),

"mailEnabled": g.get("mailEnabled"),

"memberCount": len(members),

"members": [

{

"displayName": m.get("displayName"),

"userPrincipalName": m.get("userPrincipalName"),

}

for m in members[:20] # cap at 20 to keep payload manageable

],

})

print(f" Found {len(enriched)} groups")

return enriched


# -----

# Azure OpenAI analysis

# -----

```

```
SYSTEM_PROMPT = textwrap.dedent("""
```

You are a senior Microsoft Entra ID (Azure AD) security architect.

You are given a JSON snapshot of a tenant's identity configuration pulled from Microsoft Graph API.

Your task:

1. Assess the overall security posture of the tenant.
2. Identify specific misconfigurations, risks, or gaps in each area:
 - Users (stale accounts, external users, password hygiene)
 - Privileged roles (over-privileged accounts, orphaned assignments)
 - Conditional Access policies (coverage gaps, disabled policies, missing MFA enforcement)
 - MFA registration (users without MFA, weak default methods)
 - Groups (security groups, potential for privilege escalation)
3. For each finding, rate its severity: CRITICAL / HIGH / MEDIUM / LOW
4. Provide concrete, actionable remediation steps.

Format your response with clear sections:

Executive Summary

Findings by Category

Users

Privileged Roles

```
### Conditional Access
```

```
### MFA Registration
```

```
### Groups
```

```
## Top Priority Actions
```

Be specific – reference actual user principal names, policy names, or role names from the data where relevant.

```
""").strip()
```

```
def analyze_with_openai(tenant_data: dict) -> None:
```

```
    _section("Sending data to Azure OpenAI for analysis")
```

```
    client = AzureOpenAI(
```

```
        azure_endpoint=A0AI_ENDPOINT,
```

```
        api_key=A0AI_API_KEY,
```

```
        api_version=A0AI_API_VERSION,
```

```
    )
```

```
# Serialize snapshot; truncate very large arrays to keep within token limits
```

```
payload = json.dumps(tenant_data, indent=2, default=str)
```

```
# Warn if payload is very large
```

```
token_estimate = len(payload) // 4
```

```

print(f" Payload size: ~{token_estimate:,} tokens (estimate)")

if token_estimate > 80_000:

print(" [!] Payload is large – consider reducing $select fields or tenant
size")


print(" Streaming response from Azure OpenAI...\n")


_section("Security Posture Analysis Report")


stream = client.chat.completions.create(

model=A0AI_DEPLOYMENT,

messages=[

{"role": "system", "content": SYSTEM_PROMPT},

{

"role": "user",

"content": (

"Here is the Entra ID tenant snapshot. "

"Please perform a full security posture analysis.\n\n"

f"```json\n{payload}\n```"

),

},

],

stream=True,

```

```
temperature=0.2,  
  
max_tokens=4096,  
  
)  
  
for chunk in stream:  
  
    delta = chunk.choices[0].delta if chunk.choices else None  
  
    if delta and delta.content:  
  
        print(delta.content, end="", flush=True)  
  
  
print("\n") # newline after streamed output
```

```
# -----  
  
# Main  
  
# -----
```

```
def main() -> None:  
  
    print("\n=====")  
  
    print("|| Entra ID Security Posture Analyzer ||")  
  
    print("=====")
```

```
_check_env()
```

```
# 1. Authenticate to Graph
```

```
token = _get_graph_token()

# 2. Collect tenant data

tenant_data: dict[str, Any] = {

    "users": collect_users(token),

    "directoryRoles": collect_directory_roles(token),

    "conditionalAccess": collect_conditional_access(token),

    "mfaRegistration": collect_mfa_registration(token),

    "groups": collect_groups(token),

}

# 3. Print a brief summary before sending to AI

_section("Collection Summary")

print(f" Users collected: {len(tenant_data['users'])}")

print(f" Active roles: {len(tenant_data['directoryRoles'])}")

print(f" CA policies: {len(tenant_data['conditionalAccess'])}")

print(f" MFA records: {len(tenant_data['mfaRegistration'])}")

print(f" Groups: {len(tenant_data['groups'])}")

# 4. Analyze with Azure OpenAI

analyze_with_openai(tenant_data)

print("Done.\n")
```

```
if __name__ == "__main__":
```

```
    main()
```